Laboratorio 3 de herencia

TecniHogar · Programación Orientada a Objetos · UVG

La solución permite cotizar servicios, registrar cobros y organizar visitas de cuadrillas desde una consola Java. La herencia reúne los datos comunes y el polimorfismo deja las reglas de cada servicio en su propia clase. Este informe presenta el análisis, los estados, el diseño y la evidencia de ejecución.

1 Análisis de requisitos

Información que se conserva

Entidad	Datos en memoria
Orden	Código único, cliente, dirección, horas estimadas positivas, zona, estado y visitas realizadas. La instalación agrega puntos; la certificada agrega materiales; el aire agrega BTU.
Cuadrilla	Código único, capacidad positiva por jornada e indicación de certificación en refrigeración.
Jornada	Código único, cuadrilla, órdenes asignadas, estado y resultados de las visitas. Se conserva el resultado histórico por código.
Ingresos	Dos acumuladores por categoría: cobros al registrar y cargos por visitas adicionales. No se suman costos cotizados ni los datos iniciales.

Operaciones y condiciones de validez

Operación	Condición y efecto
Cotizar	Crear temporalmente un servicio válido y mostrar costo y días. No agrega órdenes, no reserva códigos y no registra ingresos.
Registrar orden	Cliente acepta. Código de orden no repetido, servicio válido y nuevo. Se guarda pendiente, con cero visitas, y se cobra una vez el costo completo.
Registrar cuadrilla	Código no vacío ni repetido y capacidad positiva. La certificación se captura como sí o no.
Crear jornada	Código nuevo y cuadrilla existente sin jornadas abiertas. Empieza en preparación.
Asignar	Jornada en preparación y orden pendiente con visitas disponibles. Validar capacidad estimada, compatibilidad y ausencia de otra asignación.
Retirar	Solo de su jornada en preparación. Devuelve la orden a pendiente, libera horas y no consume visitas ni genera cobros.
Iniciar	Jornada en preparación y no vacía. Todas las órdenes pasan a atención y aumentan sus visitas. Cobrar Q40 por orden si esta no es su primera visita.
Registrar resultado	Jornada en curso, orden incluida y resultado aún no registrado. Completar o reprogramar/cancelar; cerrar jornada al recibir todos los resultados.
Consultar y reportar	Mostrar órdenes, cuadrillas, jornadas abiertas, capacidad disponible, conteos por estado y categoría e ingresos separados por origen y categoría.

2 Reglas y decisiones por categoría

Las horas estimadas siempre deben ser positivas y finitas. Los puntos son enteros positivos, los materiales deben tener un valor positivo y los BTU deben estar entre 9000 y 60000, incluidos los extremos. Cliente, dirección y códigos no pueden estar vacíos. La interfaz rechaza categorías, zonas, confirmaciones y números mal escritos sin terminar el programa.

Categoría y clase	Costo	Días metro / periferia	Visitas
Diagnóstico / Diagnostico	Q150 metropolitana; Q225 periférica. Hasta 1.5 horas.	1 / 2	3
Instalación / Instalacion	Base Q100 / Q175 + Q90 × ceil(max(horas estimadas, puntos × 0.75)).	3 / 5	3
Eléctrica / InstalacionElectricaCertificada	Costo de Instalacion + Q120 + 5 % de materiales.	5 / 7	2
Aire / MantenimientoAire	Q100 + Q110 × horas estimadas + Q80 de gas. Solo metropolitana; no redondea las horas.	2 / no admitida	1

Cada clase concreta sobrescribe calcularCosto(), diasAtencion(), maxVisitas() y categoria(). InstalacionElectricaCertificada utiliza super.calcularCosto() y super.diasAtencion() para ampliar la instalación. El límite de horas de diagnóstico, los puntos, los materiales y los BTU se validan en los constructores responsables de esos datos.

OrdenServicio.compatible() acepta cualquier cuadrilla existente. MantenimientoAire lo sobrescribe para exigir isRefrigeracion(). Jornada consulta ese método antes de asignar: conoce la operación de compatibilidad, pero no necesita conocer la categoría concreta. La tarifa base compartida está en OrdenServicio.tarifaBase().

Reglas de ingresos y rechazo

Sistema.registrarOrden() valida la unicidad antes de guardar y sumar el importe, redondeado a centavos con HALF_UP. Los objetos temporales de cotización nunca se incorporan a los registros. Los ocho ejemplos iniciales se agregan con cobrar=false y conservan ingresos Q0.00.

Jornada.iniciar() valida primero todas sus órdenes. Después aumenta las visitas y devuelve los cargos agrupados por categoría; Sistema.iniciar() los suma a los ingresos por visitas. Retirar o volver a asignar no cobra. Repetir el inicio se rechaza antes de cambiar visitas o ingresos. No hay devoluciones automáticas al cancelar.

Cada operación comprueba sus condiciones antes de alterar las colecciones o los estados. Las vistas de las colecciones son de solo lectura. No existen setters públicos para estado ni visitas. Se usan BigDecimal para los importes y para sumar capacidad; así 0.1 + 0.2 horas cabe exactamente en una capacidad de 0.3.

Supuestos aplicados

Los códigos son únicos dentro de cada entidad y distinguen mayúsculas; se recortan los espacios exteriores. El límite cancela únicamente cuando falla la última visita: completar esa visita deja la orden completada. Una orden fallida puede reprogramarse con otra cuadrilla libre aunque la jornada anterior aún espere otros resultados. No se incluye una fecha porque el caso define jornadas por código y no solicita calendario.

3 Estados y transiciones

Orden antes	Operación	Orden después
No registrada	Registrar y cobrar; o cargar ejemplo sin cobro	PENDIENTE, visitas = 0
PENDIENTE	Asignar a jornada válida en preparación	ASIGNADA; visitas no cambian
ASIGNADA	Retirar durante preparación	PENDIENTE; sin cobro
ASIGNADA	Iniciar jornada	EN_ATENCION; visitas + 1; Q40 desde visita 2
EN_ATENCION	Resultado completada	COMPLETADA
EN_ATENCION	Resultado fallida y quedan visitas	PENDIENTE
EN_ATENCION	Resultado fallida y agotó visitas	CANCELADA
COMPLETADA o CANCELADA	Intentar asignar o reabrir	Rechazo; permanece igual

Jornada antes	Operación	Jornada después
No creada	Crear con cuadrilla libre	PREPARACION
PREPARACION	Asignar o retirar órdenes válidas	PREPARACION
PREPARACION	Iniciar con al menos una orden	EN_CURSO
EN_CURSO	Registrar resultado y faltan otros	EN_CURSO
EN_CURSO	Registrar último resultado	CERRADA; cuadrilla libre
CERRADA	Intentar asignar, iniciar o repetir resultado	Rechazo; permanece cerrada

Una cuadrilla puede tener muchas jornadas históricas, pero solo una sin cerrar. Una orden puede aparecer en varias jornadas históricas; su estado ASIGNADA o EN_ATENCION impide una segunda asignación activa. Los resultados históricos se guardan en la jornada y no se deducen del estado actual de la orden.

Ejemplo de visitas

E01 se asigna a J1 y empieza su primera visita sin cargo adicional. Si falla, vuelve a pendiente. Al iniciar J2 se registra la segunda visita y Q40. Si falla de nuevo queda cancelada; si se completa queda completada. El total pagado al registrar no se vuelve a cobrar.

4 Clases y construcción de objetos

Clase	Responsabilidad
OrdenServicio abstracta	Conservar identidad, cliente, dirección, horas, zona, estado y visitas. Definir el contrato polimórfico y controlar transiciones comunes.
Diagnostico	Aplicar límite de 1.5 horas, tarifa fija por zona, plazo y máximo de tres visitas.
Instalacion	Guardar puntos y calcular horas facturables, costo, plazo y tres visitas.
InstalacionElectricaCertificada	Guardar materiales y ampliar costo y plazo de instalación; permitir dos visitas.
MantenimientoAire	Guardar BTU; validar zona y rango. Calcular gas y horas; exigir refrigeración y una visita.
Cuadrilla	Guardar código, capacidad y certificación, sin duplicar una bandera de ocupación.
Jornada	Administrar asignaciones, capacidad, inicio, resultados y cierre. Solicitar compatibilidad a cada orden.
Sistema	Centralizar unicidad, búsqueda, cuadrillas disponibles, operaciones, datos iniciales y reportes. Registrar ingresos.
Main	Contener main, mostrar menú, capturar entradas, crear la categoría solicitada y mostrar errores.
Validacion y enumeraciones	Validar datos comunes; limitar zona y estados a valores explícitos.

Los constructores reciben por parámetros los datos obligatorios. OrdenServicio recibe código, cliente, dirección, horas y zona; asigna pendiente y cero visitas. Diagnostico llama a super y valida el límite. Instalacion llama a super y luego valida y guarda puntos. La certificada llama al constructor de Instalacion, que a su vez llama a OrdenServicio, y finalmente valida materiales. El aire llama a OrdenServicio y valida BTU y zona.

Cuadrilla recibe sus tres datos y queda inmutable. Jornada recibe código y cuadrilla, crea sus colecciones vacías y empieza en preparación. Sistema construye mapas vacíos; Main crea un Sistema y carga los ejemplos una sola vez. Los campos privados protegen el estado; final indica que la referencia o el valor inicial no se sustituye después de construir.

Justificación de la herencia y crecimiento

Una instalación eléctrica certificada es una instalación: conserva puntos, horas facturables y tarifa, y añade certificado y garantía. Por eso extiende Instalacion. La alternativa de heredar directamente de OrdenServicio duplicaría las reglas de instalación. Componer un objeto Instalacion permitiría cambiar la forma de cálculo con más libertad, pero agregaría delegación y coordinación de datos para este ejercicio.

Para una categoría independiente se crea una subclase de OrdenServicio y se implementan costo, días, visitas y nombre; se sobrescribe compatible() si necesita restricciones. Si especializa una instalación puede extender Instalacion. Se añade su opción y captura de datos en Main.leerOrden() y se incorporan pruebas. Sistema, Jornada y los reportes no cambian: usan el contrato común y agrupan dinámicamente por categoria(). Un nuevo requisito de cuadrilla que no pueda expresarse con los datos actuales sí necesitaría ampliar Cuadrilla y su captura.

5 Evidencia de pruebas ejecutadas

Se compiló y ejecutó con OpenJDK 17.0.20. La clase test/Pruebas.java terminó sin errores y registró 133 comprobaciones aprobadas. Cada fila siguiente contiene entrada, expectativa y resultado real; se resumen con “igual” las salidas coincidentes para facilitar su lectura. El TSV incluido conserva las 133 filas completas.

Costos y tiempos

Entrada	Esperado	Obtenido
D01 metro; 1 h	Q150.00; 1 días	Q150.00; 1 días
D02 periferia; 1.5 h	Q225.00; 2 días	Q225.00; 2 días
I01 metro; 2.5 h; 5 puntos	Q460.00; 3 días	Q460.00; 3 días
I02 periferia; 1.2 h; 1 punto	Q355.00; 5 días	Q355.00; 5 días
E01 metro; 2.5 h; 5 puntos; Q2000	Q680.00; 5 días	Q680.00; 5 días
E02 periferia; 2 h; 2 puntos; Q1000	Q525.00; 7 días	Q525.00; 7 días
A01 metro; 1.5 h; 9000 BTU	Q345.00; 2 días	Q345.00; 2 días
A02 metro; 2 h; 60000 BTU	Q400.00; 2 días	Q400.00; 2 días
Instalación metro; 2.1 h; 1 punto	Q370.00; 3 días	Q370.00; 3 días
Instalación metro; 3 h; 4 puntos	Q370.00; 3 días	Q370.00; 3 días

I01 factura ceil(max(2.5, 5 × 0.75)) = 4 horas: 100 + 4 × 90 = Q460. E01 agrega 120 + 2000 × 0.05 = Q220 y llega a Q680. I02 demuestra que 1.2 horas se factura como 2. A01 conserva las 1.5 horas sin redondearlas: 100 + 165 + 80 = Q345.

Registro y cotización

Entrada o acción	Esperado	Obtenido
Inicio: 8 órdenes, 3 cuadrillas	Todas pendientes; Q0.00	Igual
Cotizar COT, diagnóstico metro 1 h	Sin agregar ni cobrar	Datos e ingresos intactos
Registrar N01, diagnóstico metro 1 h	Q150.00 de registro	Q150.00
Registrar N01 nuevamente	Rechazo sin cambios	Rechazo; estado e ingresos intactos

6 Jornadas y políticas de visitas

Entrada o secuencia	Esperado	Obtenido
J1 con C02 sin refrigeración; asignar A01	Rechazo	Rechazo; sin cambios
I01 en J1 de 6 h	Quedan 3.5 h	3.5 h
I01 asignada; intentar asignarla a J3	Rechazo	Rechazo; sin cambios
I01 + E01 + D02 en J1 de 6 h	6.5 h; rechazar D02	Rechazo; sin cambios
I01 + E01 + D01 en J1	6 h exactas; aceptar	Capacidad disponible 0
Retirar D01 antes de iniciar	Pendiente; sin ingreso	Pendiente; ingreso intacto
Iniciar J1 con I01 y E01	En atención, visita 1; Q0	Igual
Completar I01 en J1	J1 sigue en curso	EN_CURSO
Fallar E01, último resultado de J1	E01 pendiente; J1 cerrada	Igual
Asignar E01 a J4; retirar y reasignar	Q0 de visitas todavía	Q0.00
Iniciar J4 con E01	Visita 2; Q40	Q40.00
Fallar E01 por segunda vez	Cancelada	CANCELADA
0.1 h + 0.2 h; cuadrilla 0.3 h	Asignación exacta aceptada	Disponible 0

Orden y fallos sucesivos	Estados esperados	Obtenidos
D01; 3 fallos	Pendiente, pendiente, cancelada	Iguales; 3 visitas
I01; 3 fallos	Pendiente, pendiente, cancelada	Iguales; 3 visitas
E01; 2 fallos	Pendiente, cancelada	Iguales; 2 visitas
A01; 1 fallo	Cancelada	Igual; 1 visita
Recargos de esas cuatro secuencias	Q80 + Q80 + Q40 + Q0	Q200.00 total
Órdenes de ejemplo tras todas las visitas	Ingreso de registro Q0	Q0.00

Cada jornada de la secuencia quedó cerrada al registrar su único resultado; C01 pudo usarse en la jornada siguiente. La prueba también verificó que el reporte mostrara Q80.00 por visitas de diagnóstico y Q0.00 por registro de esas órdenes iniciales.

7 Rechazos y recuperación de consola

Para cada operación inválida se tomó una representación del sistema antes y después: reporte de ingresos y conteos, órdenes, cuadrillas, jornadas, asignaciones y resultados. En todos los casos se obtuvo rechazo y una representación idéntica. Las filas agrupan entradas que fueron ejecutadas por separado.

Entradas rechazadas	Esperado	Obtenido
Código de orden vacío o repetido; cuadrilla repetida	Rechazar y conservar todo	Igual
Horas 0, -1, NaN o infinito; diagnóstico 1.6 h	Rechazar y conservar todo	Igual
Puntos 0; materiales Q0; capacidad 0	Rechazar y conservar todo	Igual
BTU 8999 o 60001; aire periférico	Rechazar y conservar todo	Igual
Segunda jornada abierta para C02; código de jornada repetido	Rechazar y conservar todo	Igual
Iniciar jornada vacía o iniciar dos veces	Rechazar y conservar todo	Igual
Resultado antes de iniciar, repetido o de una orden ajena	Rechazar y conservar todo	Igual
Retiro desde otra jornada o después de iniciar	Rechazar y conservar todo	Igual
Asignar después de iniciar; asignar completada o cancelada	Rechazar y conservar todo	Igual
Orden, cuadrilla o jornada inexistente	Rechazar y conservar todo	Igual

Sesión real del menú

Entrada enviada a Main: abc → 11 → 3 → C99 → no-numero → 11 → 0. Al ingresar abc se mostró “Ingrese un número entero válido”. Al intentar registrar C99 con capacidad no-numero se mostró “Ingrese un número decimal válido usando punto”. El menú reapareció después de cada error y ambos reportes conservaron ingresos Q0.00. La opción 0 terminó normalmente.

La salida íntegra está en docs/prueba-consola.txt. Las comprobaciones se repiten con:

javac -encoding UTF-8 -d bin src/*.java test/*.java
java -cp bin Pruebas

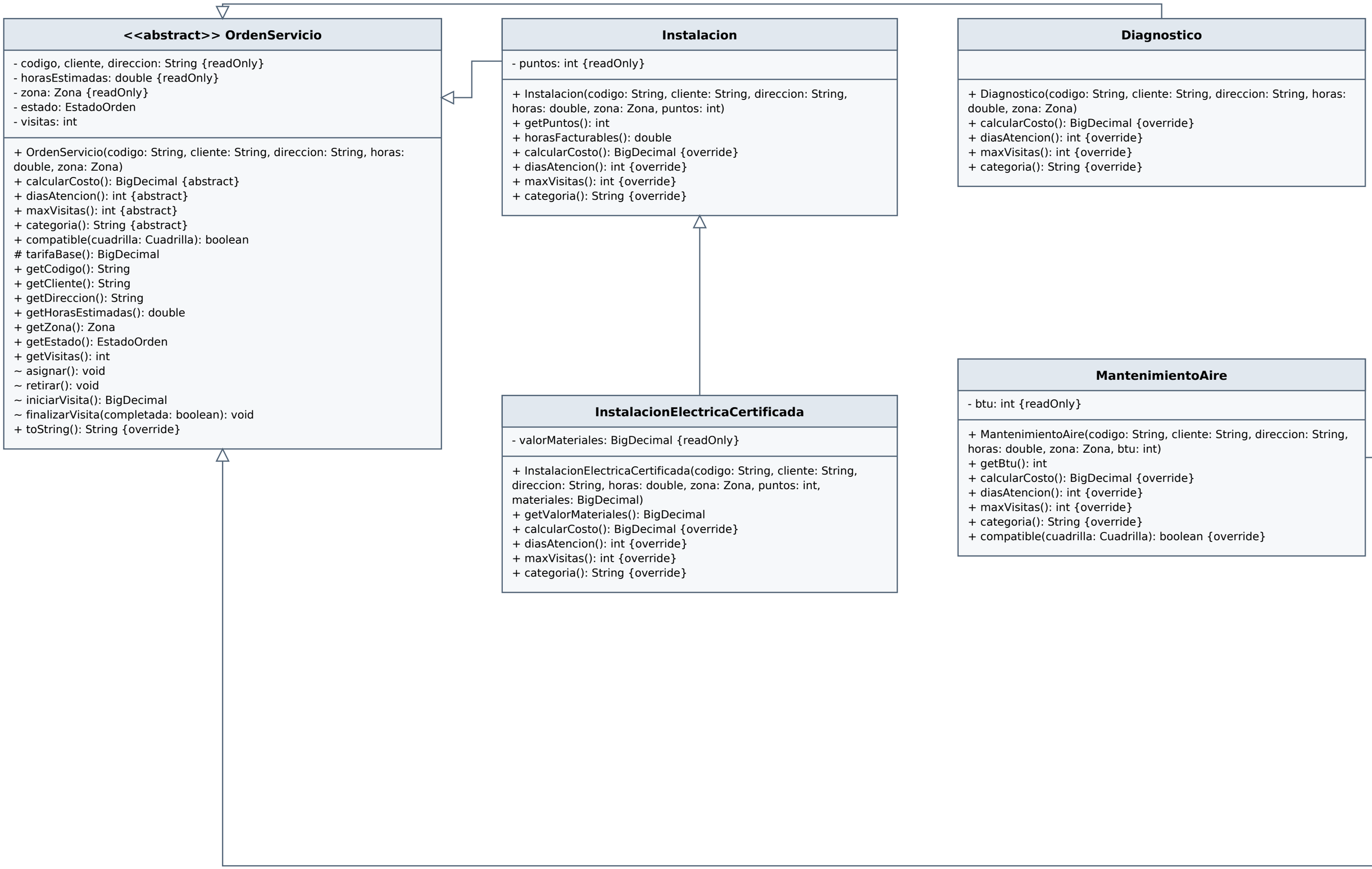
Archivos y entrega

El proyecto no requiere dependencias ni persistencia. src contiene las clases, test las pruebas y bin se genera al compilar. docs conserva el PDF, UML editable y evidencia. El README explica compilación, uso y publicación. Para Canvas se entrega este PDF y la URL del repositorio público una vez subido a GitHub.

Fuente de requisitos: Universidad del Valle de Guatemala, CC2008, “Lab 3 - Herencia”, ciclo 2 de 2026, páginas 1 a 7. Los diagramas y el catálogo de miembros que siguen corresponden a los archivos Java entregados.

UML 1 Jerarquía de servicios

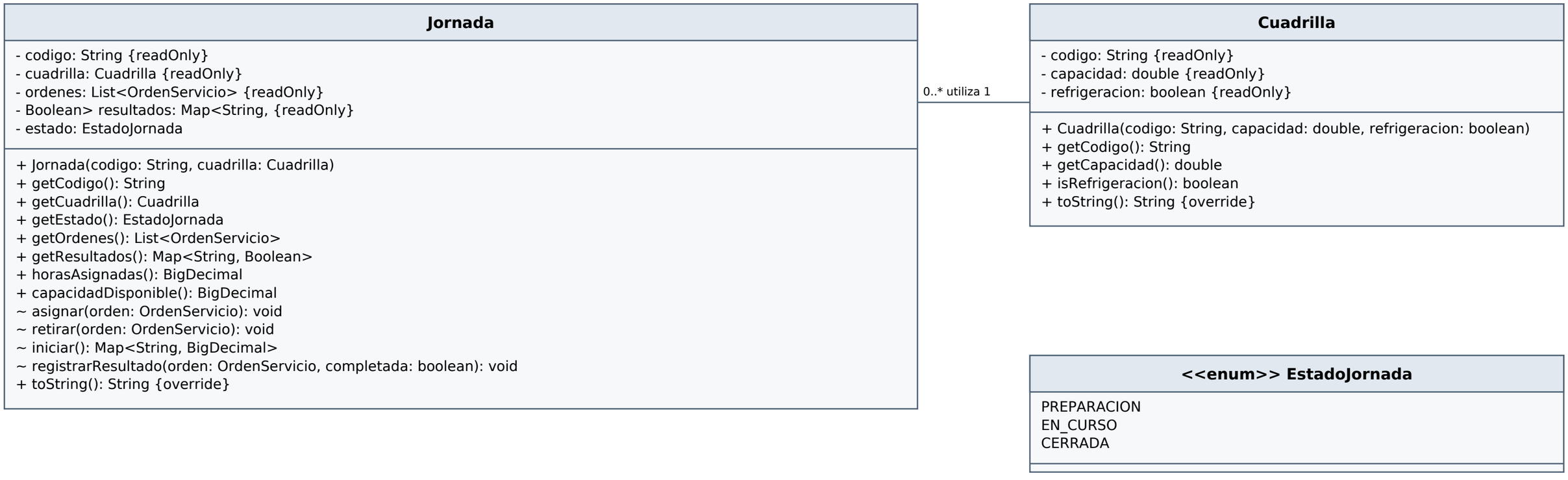
+ público - privado # protegido ~ paquete {override} sobrescrito abstract = contrato sin implementación



Los constructores validan sus datos. InstalacionElectricaCertificada reutiliza los cálculos de Instalacion mediante super.

UML 2 Jornadas y cuadrillas

La relación con órdenes es histórica: una orden puede participar en distintas jornadas a lo largo de sus visitas.



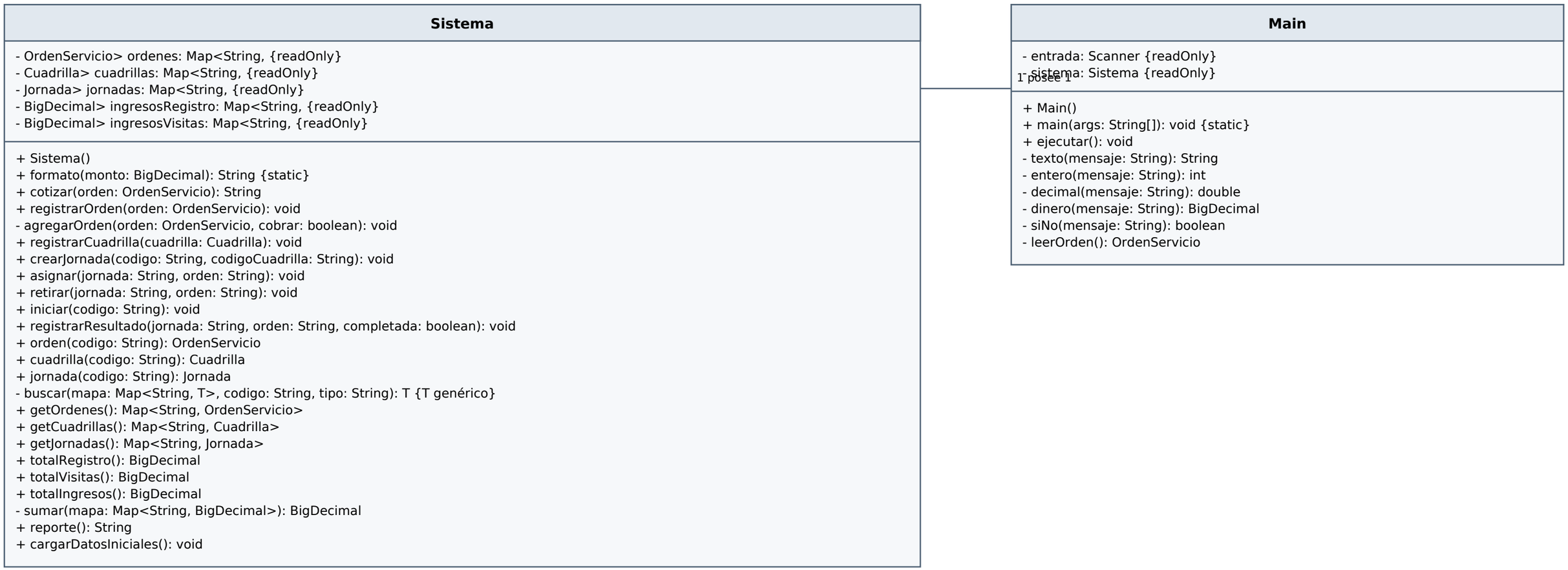
Jornada 0..* − 0..* OrdenServicio: asignaciones y visitas históricas.

Restricción: máximo una asignación activa por orden y una jornada abierta por cuadrilla.

Jornada.asignar valida capacidad y orden.compatible; iniciar devuelve cargos. El estado de jornada usa EstadoJornada.

UML 3 Sistema y menú

Las colecciones pertenecen a Sistema; los reportes agrupan por categoria() sin inspeccionar el tipo del objeto.



Sistema 1 o-- 0..* OrdenServicio: órdenes registradas.

Sistema 1 o-- 0..* Cuadrilla: cuadrillas registradas.

Sistema 1 o-- 0..* Jornada: jornadas conservadas.

Main crea Sistema y usa Scanner.

Sistema.iniciar registra los cargos devueltos por Jornada.

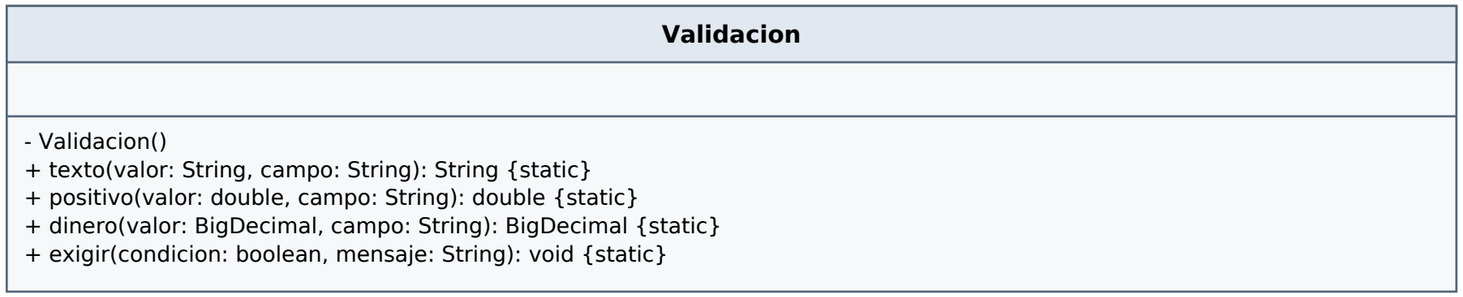
Solo registrarOrden cobra el costo inicial.

Cotizar no modifica colecciones ni ingresos.

o-- = agregación. Main.main(String[] args) es el punto de entrada. Todos los métodos declarados se muestran.

UML 4 Validaciones y dependencias

Las dependencias a enumeraciones y validaciones completan el diseño de las tres láminas anteriores.



OrdenServicio --> Zona: conserva la zona del servicio.

OrdenServicio --> EstadoOrden: conserva el estado actual.

Jornada --> EstadoJornada: conserva el estado de la jornada.

Main y las clases de dominio --> Validacion: comprueban datos.

Diagnostico, Instalacion y MantenimientoAire heredan de OrdenServicio.

InstalacionElectricaCertificada hereda de Instalacion.

Notas de diseño

Las transiciones tienen visibilidad de paquete y son usadas por Jornada.

Los constructores y datos privados se muestran en las láminas anteriores.

Los getters exponen datos o vistas sin operaciones de modificación de colecciones.

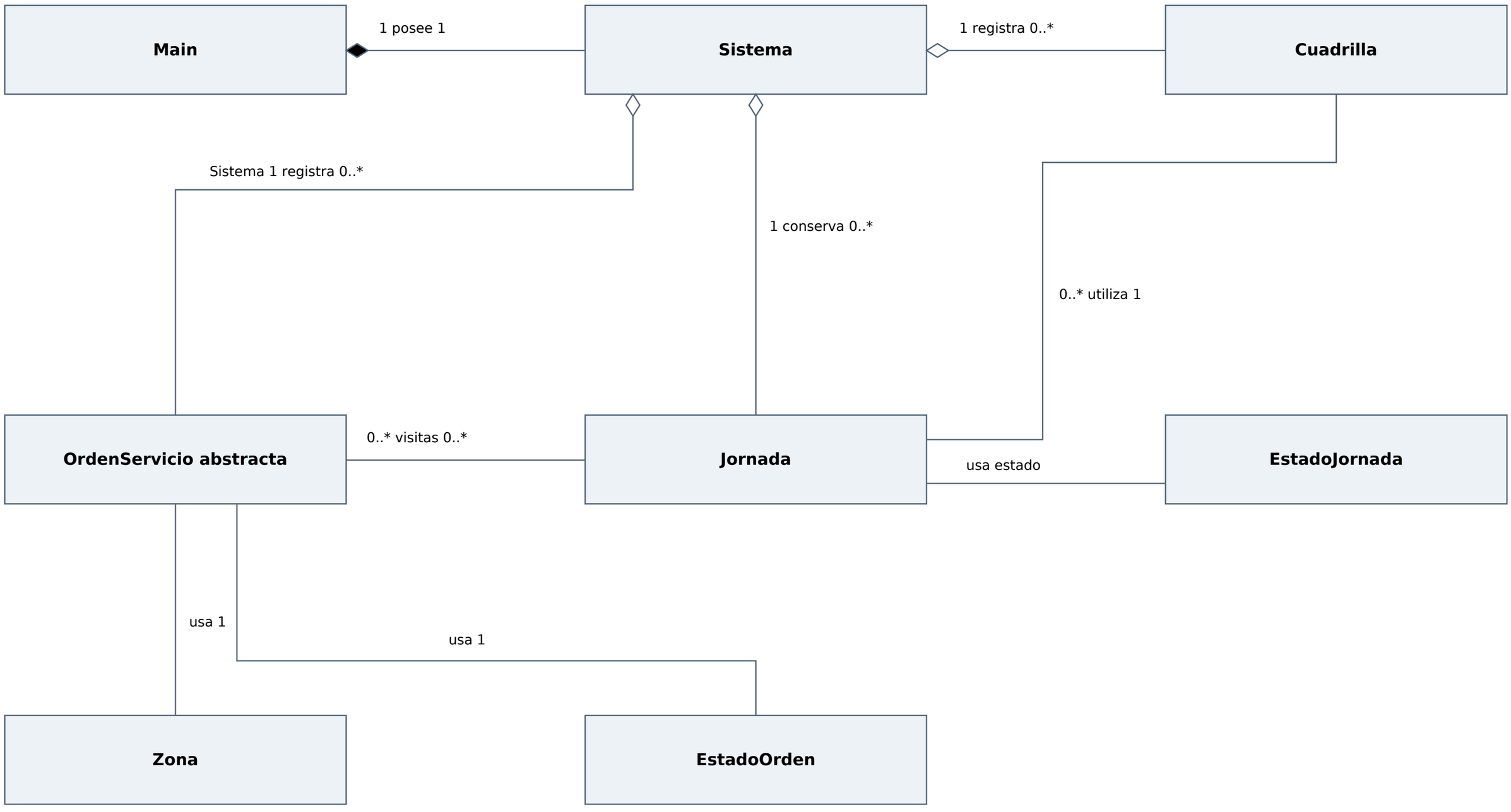
Los métodos abstractos de OrdenServicio se implementan en las categorías.

InstalacionElectricaCertificada hereda compatible() y horasFacturables().

Los detalles del propósito de cada miembro se explican en el catálogo que sigue.

UML 5 Relaciones y multiplicidades

Vista de relaciones del sistema. Los compartimentos completos están en las láminas UML 1 a 4.



Diamante lleno: composición. Diamante vacío: agregación. Líneas: asociación o dependencia indicada.

Restricciones: una jornada no cerrada por cuadrilla; una asignación activa por orden.

Catálogo de atributos y métodos

Las firmas indican visibilidad, tipo y parámetros; en un constructor no hay tipo de retorno. public es acceso público, private es privado, protected permite acceso de subclases y el signo ~ del UML indica acceso de paquete. final evita reasignar el campo; static pertenece a la clase. Los métodos void no devuelven un valor. Los parámetros conservan los nombres del código entregado.

Cuadrilla

Firma o atributo	Propósito
private final String codigo	Identificador único de la entidad.
private final double capacidad	Horas máximas de la cuadrilla por jornada.
private final boolean refrigeracion	Indica certificación en refrigeración.
public Cuadrilla(String codigo, double capacidad, boolean refrigeracion)	Constructor: valida e inicializa los datos recibidos.
public String getCodigo()	Consulta sin modificar. Identificador único de la entidad.
public double getCapacidad()	Consulta sin modificar. Horas máximas de la cuadrilla por jornada.
public boolean isRefrigeracion()	Consulta sin modificar. Indica certificación en refrigeración.
public String toString() [sobrescrito]	Devuelve una representación legible para consultas de consola.

Diagnostico

Firma o atributo	Propósito
public Diagnostico(String codigo, String cliente, String direccion, double horas, Zona zona)	Constructor: valida e inicializa los datos recibidos. Llama a super para inicializar OrdenServicio.
public BigDecimal calcularCosto() [sobrescrito]	Devuelve el costo con las reglas de Diagnostico.
public int diasAtencion() [sobrescrito]	Devuelve los días hábiles propios de la categoría.
public int maxVisitas() [sobrescrito]	Devuelve 3 visitas permitidas.
public String categoria() [sobrescrito]	Devuelve el nombre de esta categoría para consultas y reportes.

EstadoJornada

Firma o atributo	Propósito
PREPARACION, EN_CURSO, CERRADA	Valores permitidos; no admite otros textos como estado o zona.

EstadoOrden

Firma o atributo	Propósito
PENDIENTE, ASIGNADA, EN_ATENCION, COMPLETADA, CANCELADA	Valores permitidos; no admite otros textos como estado o zona.

Instalacion

Firma o atributo	Propósito
private final int puntos	Cantidad positiva de puntos a instalar.
public Instalacion(String codigo, String cliente, String direccion, double horas, Zona zona, int puntos)	Constructor: valida e inicializa los datos recibidos. Llama a super para inicializar OrdenServicio.
public int getPuntos()	Consulta sin modificar. Cantidad positiva de puntos a instalar.
public double horasFacturables()	Devuelve ceil(max(horas estimadas, puntos × 0.75)).
public BigDecimal calcularCosto() [sobrescrito]	Devuelve el costo con las reglas de Instalacion.
public int diasAtencion() [sobrescrito]	Devuelve los días hábiles propios de la categoría.
public int maxVisitas() [sobrescrito]	Devuelve 3 visitas permitidas.
public String categoria() [sobrescrito]	Devuelve el nombre de esta categoría para consultas y reportes.

InstalacionElectricaCertificada

Firma o atributo	Propósito
private final BigDecimal valorMateriales	Valor positivo declarado para calcular la garantía.
public InstalacionElectricaCertificada(String codigo, String cliente, String direccion, double horas, Zona zona, int puntos, BigDecimal materiales)	Constructor: valida e inicializa los datos recibidos. Llama a super para inicializar Instalacion.
public BigDecimal getValorMateriales()	Consulta sin modificar. Valor positivo declarado para calcular la garantía.
public BigDecimal calcularCosto() [sobrescrito]	Devuelve el costo con las reglas de InstalacionElectricaCertificada.
public int diasAtencion() [sobrescrito]	Devuelve los días hábiles propios de la categoría.
public int maxVisitas() [sobrescrito]	Devuelve 2 visitas permitidas.
public String categoria() [sobrescrito]	Devuelve el nombre de esta categoría para consultas y reportes.

Jornada

Firma o atributo	Propósito
private final String codigo	Identificador único de la entidad.
private final Cuadrilla cuadrilla	Cuadrilla que atiende la jornada.
private final List<OrdenServicio> ordenes	Órdenes registradas o asignadas según la clase.
private final Map<String, Boolean> resultados	Resultado histórico por código; true completada, false fallida.
private EstadoJornada estado	Estado actual protegido de la entidad.
public Jornada(String codigo, Cuadrilla cuadrilla)	Constructor: valida e inicializa los datos recibidos. Inicia en preparación con colecciones vacías.
public String getCodigo()	Consulta sin modificar. Identificador único de la entidad.
public Cuadrilla getCuadrilla()	Consulta sin modificar. Cuadrilla que atiende la jornada.
public EstadoJornada getEstado()	Consulta sin modificar. Estado actual protegido de la entidad.
public List<OrdenServicio> getOrdenes()	Consulta sin modificar. Órdenes registradas o asignadas según la clase. Devuelve vista no modificable.
public Map<String, Boolean> getResultados()	Consulta sin modificar. Resultado histórico por código; true completada, false fallida. Devuelve vista no modificable.
public BigDecimal horasAsignadas()	Suma horas estimadas de las órdenes.
public BigDecimal capacidadDisponible()	Devuelve capacidad menos horas asignadas.
void asignar(OrdenServicio orden)	Valida preparación, disponibilidad, compatibilidad y capacidad; asigna.
void retirar(OrdenServicio orden)	Retira una orden propia durante preparación y la libera.
Map<String, BigDecimal> iniciar()	Prevalida todo, inicia visitas y devuelve cargos por categoría.
void registrarResultado(OrdenServicio orden, boolean completada)	Guarda resultado único y cierra al recibir todos.
public String toString() [sobrescrito]	Devuelve una representación legible para consultas de consola.

Main

Firma o atributo	Propósito
private final Scanner entrada	Lector de consola.
private final Sistema sistema	Sistema administrado por el menú.
public Main()	Constructor: inicializa el sistema y carga los datos de ejemplo.
public static void main(String[] args)	Punto de entrada; crea Main y ejecuta el menú.
public void ejecutar()	Repite menú hasta salir; captura errores y fin de entrada.
private String texto(String mensaje)	Muestra mensaje y lee texto no vacío.
private int entero(String mensaje)	Lee entero o informa formato inválido.
private double decimal(String mensaje)	Lee double positivo y finito.
private BigDecimal dinero(String mensaje)	Lee importe positivo en BigDecimal.
private boolean siNo(String mensaje)	Acepta únicamente s/n y devuelve boolean.
private OrdenServicio leerOrden()	Captura datos y construye la categoría elegida.

MantenimientoAire

Firma o atributo	Propósito
private final int btu	Capacidad del equipo entre 9000 y 60000.
public MantenimientoAire(String codigo, String cliente, String direccion, double horas, Zona zona, int btu)	Constructor: valida e inicializa los datos recibidos. Llama a super para inicializar OrdenServicio.
public int getBtu()	Consulta sin modificar. Capacidad del equipo entre 9000 y 60000.
public BigDecimal calcularCosto() [sobrescrito]	Devuelve el costo con las reglas de MantenimientoAire.
public int diasAtencion() [sobrescrito]	Devuelve los días hábiles propios de la categoría.
public int maxVisitas() [sobrescrito]	Devuelve 1 visitas permitidas.
public String categoria() [sobrescrito]	Devuelve el nombre de esta categoría para consultas y reportes.
public boolean compatible(Cuadrilla cuadrilla) [sobrescrito]	Devuelve true solo si la cuadrilla existe y tiene refrigeración.

OrdenServicio abstracta

Firma o atributo	Propósito
private final String codigo, cliente, direccion	Identificador único de la entidad. Nombre del cliente. Dirección de la visita.
private final double horasEstimadas	Horas reales previstas usadas para capacidad.
private final Zona zona	Zona para tarifas y plazos.
private EstadoOrden estado	Estado actual protegido de la entidad.
private int visitas	Número de visitas ya iniciadas.
public OrdenServicio(String codigo, String cliente, String direccion, double horas, Zona zona)	Constructor: valida e inicializa los datos recibidos.
public abstract BigDecimal calcularCosto()	Contrato de costo del servicio; devuelve BigDecimal.
public abstract int diasAtencion()	Contrato de plazo en días hábiles.
public abstract int maxVisitas()	Contrato de límite de visitas por categoría.
public abstract String categoria()	Nombre de categoría usado por reportes.
public boolean compatible(Cuadrilla cuadrilla)	Acepta una cuadrilla existente; punto de extensión.
protected BigDecimal tarifaBase()	Devuelve Q100 o Q175 según zona.
public String getCodigo()	Consulta sin modificar. Identificador único de la entidad.
public String getCliente()	Consulta sin modificar. Nombre del cliente.
public String getDireccion()	Consulta sin modificar. Dirección de la visita.
public double getHorasEstimadas()	Consulta sin modificar. Horas reales previstas usadas para capacidad.
public Zona getZona()	Consulta sin modificar. Zona para tarifas y plazos.
public EstadoOrden getEstado()	Consulta sin modificar. Estado actual protegido de la entidad.
public int getVisitas()	Consulta sin modificar. Número de visitas ya iniciadas.
void asignar()	Valida pendiente y cambia a ASIGNADA.
void retirar()	Valida asignada y devuelve a PENDIENTE.
BigDecimal iniciarVisita()	Aumenta visitas, pasa a atención y devuelve Q0 o Q40.
void finalizarVisita(boolean completada)	Aplica completada, pendiente o cancelada según resultado.
public String toString() [sobrescrito]	Devuelve una representación legible para consultas de consola.

Sistema

Firma o atributo	Propósito
private final Map<String, OrdenServicio> ordenes	Órdenes registradas o asignadas según la clase.
private final Map<String, Cuadrilla> cuadrillas	Cuadrillas por código.
private final Map<String, Jornada> jornadas	Jornadas por código.
private final Map<String, BigDecimal> ingresosRegistro	Importes cobrados al registrar, agrupados por categoría.
private final Map<String, BigDecimal> ingresosVisitas	Recargos de visitas adicionales por categoría.
public Sistema()	Constructor: inicializa los mapas vacíos, sin ingresos.
public static String formato(BigDecimal monto)	Convierte importe a Q y dos decimales, HALF_UP.
public String cotizar(OrdenServicio orden)	Devuelve costo y plazo sin registrar datos.
public void registrarOrden(OrdenServicio orden)	Registra una orden nueva con cobro.
private void agregarOrden(OrdenServicio orden, boolean cobrar)	Valida, guarda y cobra solo si cobrar=true.
public void registrarCuadrilla(Cuadrilla cuadrilla)	Valida unicidad y guarda la cuadrilla.
public void crearJornada(String codigo, String codigoCuadrilla)	Valida código, cuadrilla libre y crea jornada.
public void asignar(String jornada, String orden)	Busca objetos y delega asignación a Jornada.
public void retirar(String jornada, String orden)	Busca objetos y delega retiro a Jornada.
public void iniciar(String codigo)	Inicia jornada y acumula cargos por categoría.
public void registrarResultado(String jornada, String orden, boolean completada)	Busca objetos y delega el resultado a Jornada.
public OrdenServicio orden(String codigo)	Busca orden por código; rechaza si no existe.
public Cuadrilla cuadrilla(String codigo)	Busca cuadrilla por código; rechaza si no existe.
public Jornada jornada(String codigo)	Busca jornada por código; rechaza si no existe.
private <T> T buscar(Map<String, T> mapa, String codigo, String tipo)	Búsqueda genérica validada en un mapa.
public Map<String, OrdenServicio> getOrdenes()	Consulta sin modificar. Órdenes registradas o asignadas según la clase. Devuelve vista no modificable.
public Map<String, Cuadrilla> getCuadrillas()	Consulta sin modificar. Cuadrillas por código. Devuelve vista no modificable.
public Map<String, Jornada> getJornadas()	Consulta sin modificar. Jornadas por código. Devuelve vista no modificable.
public BigDecimal totalRegistro()	Devuelve suma de ingresos al registrar.
public BigDecimal totalVisitas()	Devuelve suma de cargos adicionales.
public BigDecimal totalIngresos()	Devuelve registro más visitas.
private BigDecimal sumar(Map<String, BigDecimal> mapa)	Suma importes de un mapa por categoría.
public String reporte()	Construye conteos, ingresos y jornadas abiertas.
public void cargarDatosIniciales()	Carga ocho órdenes sin cobro y tres cuadrillas en sistema vacío.

Validacion

Firma o atributo	Propósito
private Validacion()	Constructor privado: impide crear una utilidad sin estado.
public static String texto(String valor, String campo)	Rechaza nulo o vacío y devuelve texto recortado.
public static double positivo(double valor, String campo)	Rechaza cero, negativos, NaN e infinito; devuelve el valor.
public static BigDecimal dinero(BigDecimal valor, String campo)	Exige importe no nulo y positivo; devuelve el importe.
public static void exigir(boolean condicion, String mensaje)	Lanza IllegalArgumentException si la condición es falsa.

Zona

Firma o atributo	Propósito
METROPOLITANA, PERIFERICA	Valores permitidos; no admite otros textos como estado o zona.